OPERATING SYSTEM

Aarohi Jaiswal

PES1UG24CS009

4A

PHASE 1

1a

```
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1b

```
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$
```

PHASE 2

2a

```
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$
```

2b

```
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
```

PHASE 3

3a

```
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ ./pes add file1.txt file2.txt
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.h
  untracked:  Makefile
  untracked:  index.h
  untracked:  tree.c
  untracked:  test_tree
  untracked:  test_sequence.sh
  untracked:  object.c
  untracked:  commit.c
  untracked:  index.c
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  test_objects
  untracked:  commit.h
  untracked:  pes.h
```

3b

```
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776705843 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776705851 6 file2.txt
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ git push --force origin main
```

PHASE 4

4a

```
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ ./pes log
commit 3ec2743374e606e4957f82bf3ef10fec42efdd0ddcb0df93e547f14c6fa10256
Author: Aarohi Jaiswal PES1UG24CS009
Date:   1776708081

    Add farewell

commit d06172b2d2ab64d3c08e69ea6bbb810b578e4809c5e12436b5ac380d22d0c81c
Author: Aarohi Jaiswal PES1UG24CS009
Date:   1776708081

    Add world

Add farewell

commit 321586099a4a8e9fde131818aed0cf34ad88d7974e78ceecf346ce2697983f94
Author: Aarohi Jaiswal PES1UG24CS009
Date:   1776707996

    Initial commit
```

4b

```
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/32/1586099a4a8e9fde131818aed0cf34ad88d7974e78ceecf346ce2697983f94
.pes/objects/3e/c2743374e606e4957f82bf3ef10fec42efdd0ddcb0df93e547f14c6fa10256
.pes/objects/d0/6172b2d2ab64d3c08e69ea6bbb810b578e4809c5e12436b5ac380d22d0c81c
.pes/objects/d7/1b25693891507a11cd5dbc58f431b16b1ec9a907dafafac66014e1585d775e
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4c

```
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ cat .pes/refs/heads/main
3ec2743374e606e4957f82bf3ef10fec42efdd0ddcb0df93e547f14c6fa10256
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$ cat .pes/HEAD
ref: refs/heads/main
[04/20/26]seed@VM:~/PES1UG24CS009-PES-VCS$
```

Theory Question

QUESTION-5

Q5.1 A branch in pes is simply a file inside .pes refs heads that stores a commit hash so checkout mainly means switching which commit the repository is pointing to To implement pes checkout branch you first read the target branch file to get its commit hash then update the HEAD file so it points to that branch After that you must rebuild the working directory by reading the tree of that commit and writing all files and folders from the object store into the filesystem while also removing files that are no longer present Finally the index must be updated to match this new snapshot The operation is complex because it affects multiple parts at once including HEAD index and working

directory and it must carefully avoid overwriting user changes and correctly handle nested directory structures

Q5.2 To detect a dirty working directory you compare three things the working directory the index and the target commit For each tracked file you first check if the working directory version differs from the index using metadata like modification time and size If it differs it means the user has uncommitted changes Then you compare the index version with the version stored in the target commit by checking hashes If both comparisons show differences then switching branches would overwrite local changes so checkout must be refused This works because the index represents the last staged state while the object store represents committed history so you can detect conflicts without rehashing file contents

Q5.3 Detached HEAD means that HEAD points directly to a commit instead of a branch In this state when you make a new commit it is created normally but no branch points to it so it becomes an unnamed commit If you later switch to another branch these commits can become unreachable and may eventually be lost To recover them the user can create a new branch pointing to that commit or use the commit hash to check it out again The key idea is that commits still exist but without a reference they are not part of the visible history

QUESTION-6

Q6.1 Over time the object store may contain objects that are no longer referenced by any branch so garbage collection works by first finding all reachable objects and then deleting the rest The algorithm starts from every branch reference and HEAD and performs a traversal of the commit graph For each commit you visit you mark it as reachable then follow its tree and parent links and recursively mark all referenced trees and blobs A hash set is the best data structure to track reachable hashes because it allows fast insertion and lookup Once traversal is complete you scan the entire object store and delete any object whose hash is not in the reachable set For a repository with one hundred thousand commits and fifty branches you would not visit fifty times the commits because many branches share history so in practice you visit roughly the number of unique commits around one hundred thousand along with their trees and blobs which could be a few times larger depending on file structure

Q6.2 Running garbage collection at the same time as a commit is dangerous because of a race condition where a new object is created but not yet fully referenced A commit operation first writes objects and then updates references like HEAD If garbage collection runs in between it may scan the object store before the new commit is linked from any branch and mistakenly treat it as unreachable and delete it This leads to data loss or corrupted history Git avoids this by ensuring GC only runs when the repository is in a stable state and by using mechanisms like temporary files atomic renames and reachability protection so that newly created objects are not collected until they are safely referenced